Maintainer’s Manual

Future maintenance and development must prioritize UI performance and data integrity. Maintainers should ensure that all marker updates are logged within the database to preserve the audit trail shown in the UI's history log. When modifying the codebase, developers must adhere to Gerrit’s existing style guides, specifically using standard CSS variables to maintain Dark Mode compatibility and avoiding external dependencies that could bloat the diff-viewer’s footprint. Testing is mandatory for all changes; the suite includes Bazel-driven unit and integration tests that cover marker persistence, multi-user visibility, and boundary cases for line ranges. For deployment, the standard Gerrit build process should be followed, ensuring that the read-markers endpoints are optimized with proper indexing to prevent latency during the loading of large files.

Requirements Analysis and Specification

The system must support authors, reviewers, and administrators without changing Gerrit’s core review semantics. It must integrate with existing Gerrit permissions, remain compatible with normal change review workflows, and include tests that future maintainers can run to prevent regressions.

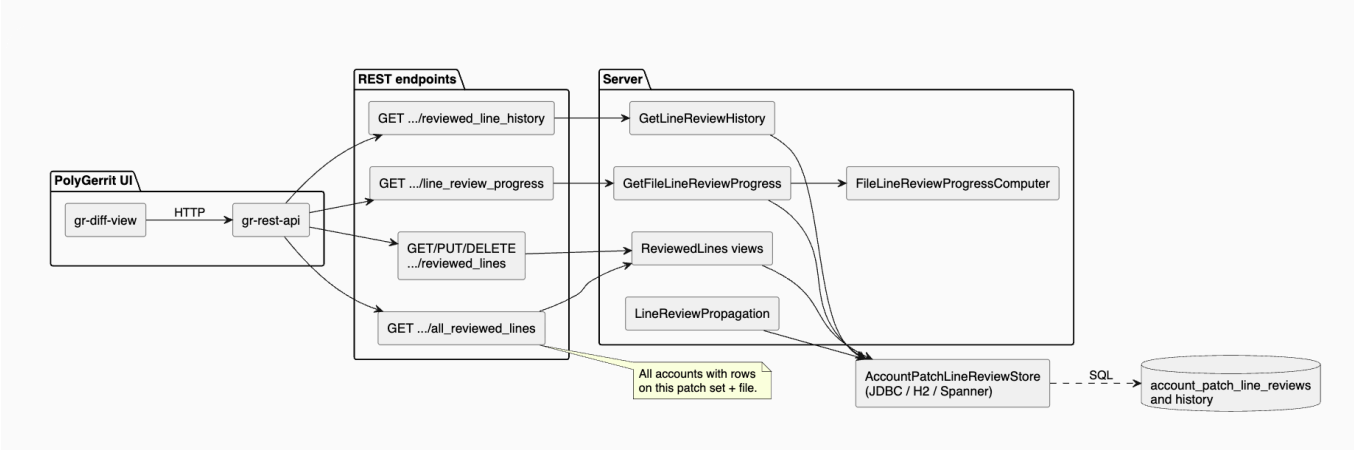
From a reviewer’s viewpoint, the system must make review tracking feel lightweight and natural during the code review process. Reviewers should be able to toggle the read status of a specific line range or region in a file with ease, using interactions that fit existing review workflows. The UI should clearly indicate read progress such that reviewers can easily understand their progress for each file and for the overall patch set, and reviewers should be able to view the progress overview to see what percentage of the lines are reviewed already. The system must also allow reviewers to access other reviewers’ read markers within the same patch set in an easily understandable manner, while still keeping each reviewer’s markers attributable and distinct, by using different colors for different reviewers’ markers for instance. Reviewers should have full control over their read markers and be able to toggle their status at any time.

From a system perspective, line and region-level markers must be represented and stored in a consistent and scalable manner. In particular, the system persists read markers per reviewer for each patch set, and it must support efficient retrieval for recovering and displaying read marker summaries without expensive recomputation. The system must also correctly handle concurrent usage, as multiple reviewers may update their own markers and view others’ markers at overlapping time intervals. The UI should remain responsive in these situations, and the backing storage should remain consistent. Since the proposed features introduces high-granularity state that is expected to persist for long periods of time, storage and history retention must be carefully managed to prevent unbounded growth.

From a client perspective, the proposed features should integrate smoothly with the existing Gerrit review and permissions models. Read markers should still be stored per reviewer, and only authenticated reviewers should be able to create or modify their own markers. The design should clearly separate concerns such as the UI, data storage, and propagation logic such that the system is maintainable.

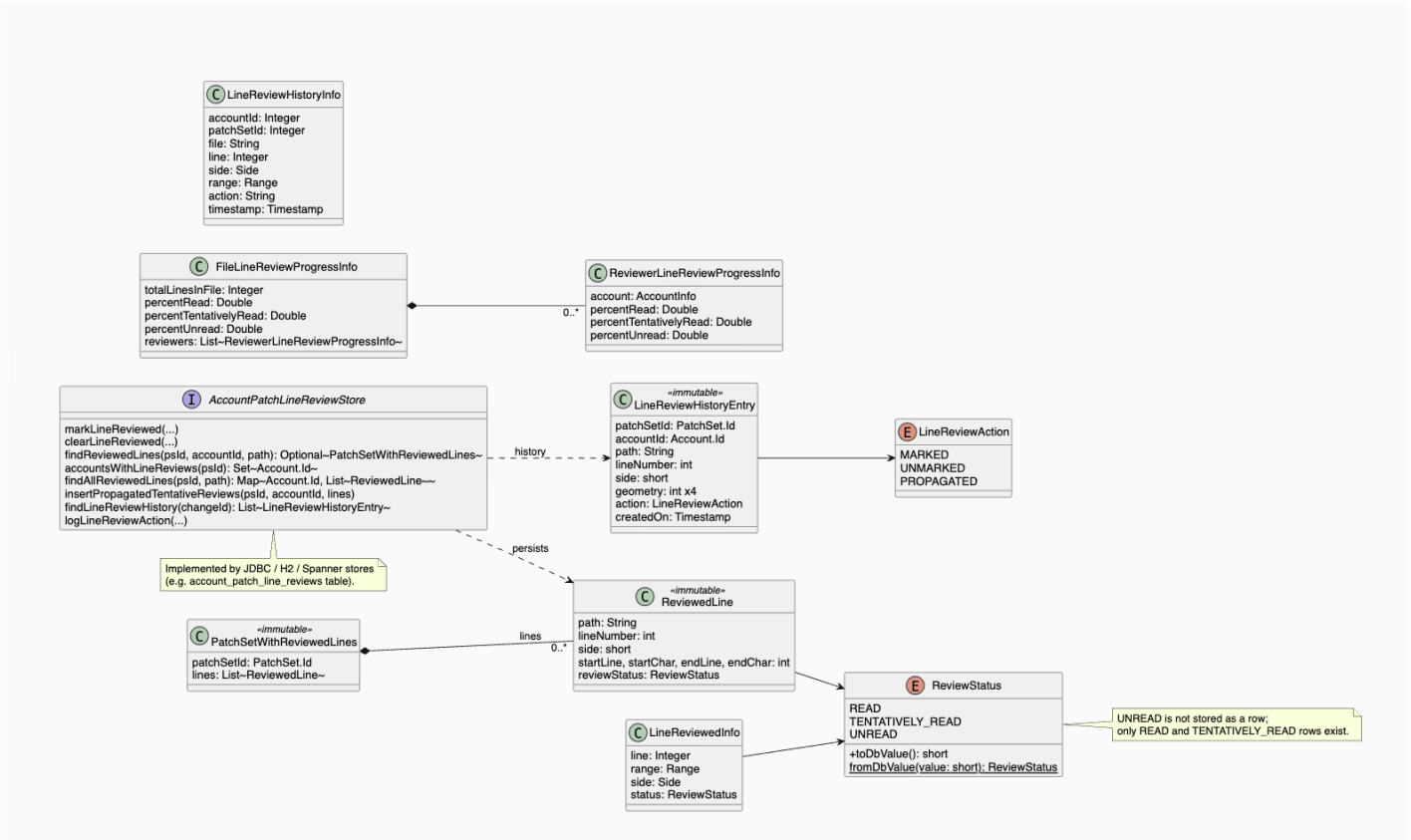
Architectural Diagram

The following shows how the frontend and backend interact with each other (via PolyGerrit-UI, REST endpoints, the Server, and the database)

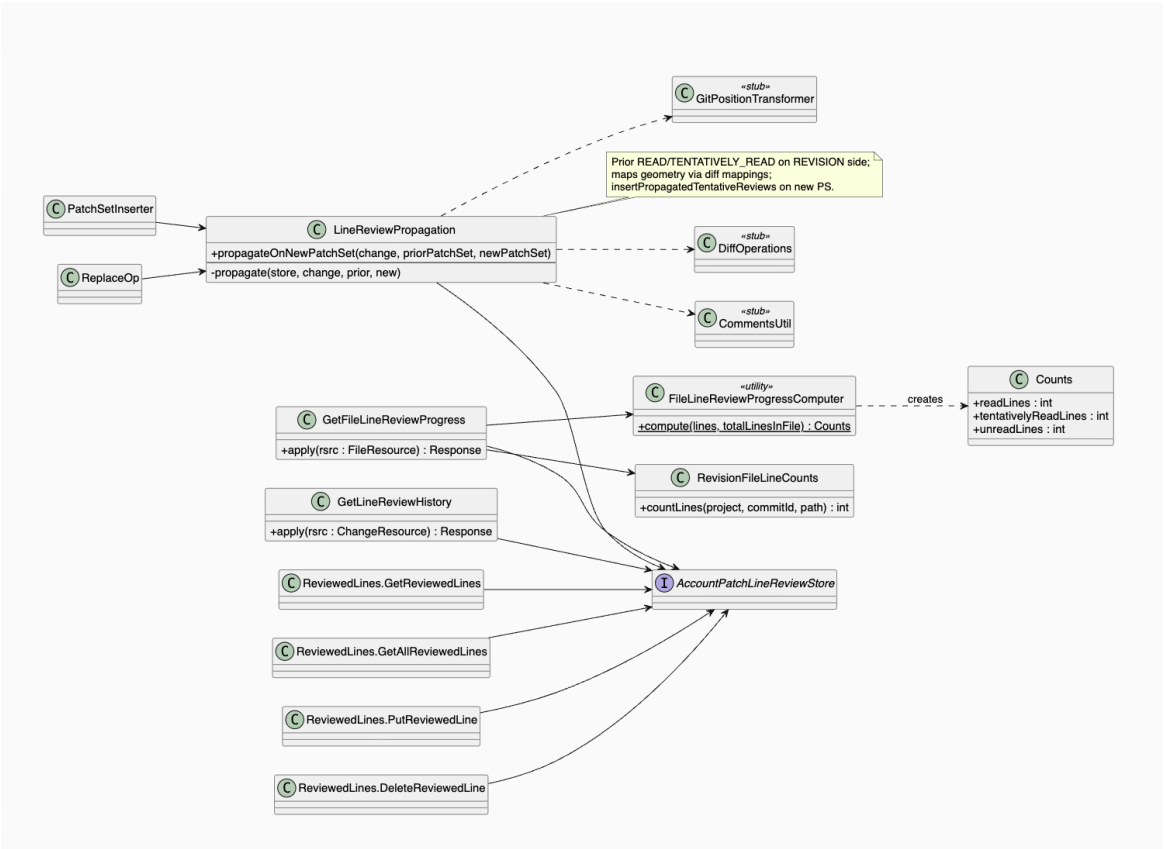


Class Diagrams

The following shows the fundamental classes that were newly created to introduce line-level review markers to Gerrit



The following shows the new classes used to implement specific features (propagation of read status as tentatively read, visibility of other reviewers' markers, review history, and summary of progress)



User Interface Design

The interface follows Gerrit's existing UI patterns. New controls are placed near the workflow they support to avoid interrupting the standard review flow and use concise labels, so they are not too cluttered or distracting. For instance, the read markers are placed next to the line that they indicate review status for.

System Design

The implementation is organized around Gerrit's existing frontend and backend structure. UI components handle user interaction. Service or API layers communicate with Gerrit data sources.

Developer Workflow

Developers should follow the host project's existing style, build, and review process. New code should be small, focused, and covered by tests. UI code should use Gerrit's established component patterns. Backend code should rely on existing APIs where possible.

1. Clone the repo locally
2. Pick a Github issue (a new feature or bug fix) to work on
3. Create a new branch for that issue/feature
4. Write code and add tests during development process
5. Submit a Pull Request and at least one other team member to review
6. Address feedback from Pull Request

7. Ensure CI/CD pipeline runs successfully
8. Merge feature branch into main branch

Style Guide
We use clear names, add comments/documentation for functions, and try to follow existing Gerrit conventions. We keep UI text concise and user-facing errors actionable. We also avoid adding new dependencies unless they are necessary and justified. Mocks/fakes are used during testing for efficiency and simplicity.

Test Facilities
Future developers should run the relevant unit, integration, and UI tests before merging changes. At minimum, tests should cover basic use, invalid input, boundary values, and regression cases discovered during development.

Example test workflow:
 # Run frontend tests
bazel test //polygerrit-ui/...
 # Run relevant backend tests
bazel test //...
 # Run a focused test target when modifying one area
bazel test //path/to:target

Regression tests should be added for every fixed bug.

Test Plan and Results
Tests include multiple unit tests per feature for core logic, UI tests for user interaction, and manual end-to-end tests for each role. The final result should show that the new capabilities work for intended users without breaking existing Gerrit behavior.

Unit tests are run using Bazel (`//javatests/com/google/gerrit/server:server_tests`). End-to-end tests run in `//javatests/com/google/gerrit/acceptance/api/revision:RevisionIT`. Testing was done incrementally, so whenever we added a new feature, we had some unit tests to address the basic functionality of the added functions and potentially some edge cases. Based on feedback, we added more tests to cover more scenarios. We also added end-to-end tests to test the whole flow of the program, including testing API endpoints. Finally, we got numeric coverage and added more tests as needed. Testing was therefore done throughout our development process.

For the frontend specifically, coverage can be measured in three buckets (state/logic coverage, interaction coverage, and visual coverage). For automatable frontend tests, we report line

coverage and branch coverage on the diff-history UI components and related logic. We can run `bazel coverage` on the frontend-facing unit tests created for our feature to determine the total coverage of our test suite, which approximates our current automated test coverage to about 95.8% of the code written to support it. Currently, the largest gap in test coverage exists for the code relating to `getReviewedLineHistory()` in `gr-rest-api-impl.ts`, since this portion of the codebase is heavy in its interactions with the review states of other users, and is difficult to simulate in the test environment.

For the feature for viewing other reviewers' line markers, running `bazel coverage` for the unit tests on `JdbcAccountPatchLineReviewStoreTest.java` indicates that we achieve approximately 70% line and function coverage, with many of the uncovered lines being those for configuring the backend for different database backends, which is not exercised by the unit tests. This coverage metric is not fully representative of our tests since it does not include our end-to-end tests, which exercise the full API stack, including HTTP routing and database reads.

For the feature for retrieving the history of review markers, we tried to merge coverage from unit tests with end-to-end tests, but our environment and `bazel coverage` setup was only able to produce a coverage report for the unit tests. The line coverage for `JdbcAccountPatchLineReviewStore.java` as measured by the unit tests was 61.6%. However, this does not fully represent the coverage from end-to-end tests, which cover the REST handler and store query path.

For the feature for propagating status of read markers, we tried to merge the coverage from unit tests with end-to-end tests, but that wouldn't work due to local Bazel/lcov reporting limitations. We only have numerical results for test coverage for two of the files relevant to the feature, with coverage being 66.5% (450 of 677 lines). However, that does not fully represent our coverage. The unit tests in `LineReviewPropagationTest` cover core functionality and edge cases: revision-side propagation, parent-side exclusion, missing commit IDs, deleted-file drops, multi-account behavior, and range/region propagation. The unit tests in `JdbcAccountPatchLineReviewStoreTest` cover important transitions/persistence (mark, clear, carryover). `RevisionIT` covers propagation through real patch set updates and API-level flows.

For the feature for summary of review progress, we tried to merge the coverage from unit tests with end-to-end tests but our environment was only able to get coverage by the unit tests for some relevant files. The line coverage (for relevant files captured) was 75.8%. The unit tests cover core functionality (correctly calculating percentage of lines that are read, tentatively read, unread). It also covers some edge cases regarding bounds of specified lines, ignoring parent-side markers, and zero file-size. The end-to-end tests cover those scenarios by exercising the API endpoints.

Deployment Procedure

1. Make changes to the code, addressing the corresponding Github issue.
2. Build the project using the normal Gerrit build process.
3. Run the relevant test suite.
4. Run locally to verify Gerrit workflow
    a. To run frontend: yarn start
    b. To run backend: $JAVA_HOME/bin/java "-DsourceRoot=$(bazel info workspace)" -jar bazel-bin/gerrit.war daemon -d $GERRIT_SITE --console-log --dev-cdn http://localhost:8080
5. Commit and push changes to the feature branch.
6. Verify CI/CD pipeline runs successfully.
7. Make Pull Request for the Github issue in preparation for merging into main branch.
8. Merge and deploy to production only after review approval and successful validation.